Тема №8. Решение оптимизационных задач на графах

8.1 Краткие теоретические сведения

Для представления и анализа различных структур, имеющих связи между входящими в них элементами применяется особая математическая конструкция, называемая графом [15].

Граф представляет собой множество точек (узлов или вершин), часть из которых или все соединены линиями, изображающими связи между этими точками. Эти линии называются ребрами.

Существует два основных вида графов: ориентированные и неориентированные. В ориентированном графе линии, соединяющие вершины, имеют направление от одной из них к другой. В неориентированном графе линии не имеют направления.

Граф называется простым, если каждую пару вершин соединяет не более чем одно ребро. Граф называется мультиграфом, если хотя бы одну пару вершин соединяет более чем одно ребро.

Граф может быть задан двумя дискретными множествами – множеством вершин, и множеством ребер.

Если две вершины соединены ребром, они называются смежными. Граф можно задать, если для всех пар вершин указать наличие или отсутствие между ними ребра. Это можно реализовать с помощью матрицы, элемент которой c_{ij} принимает значение 1, если вершина i смежна вершине j, и значение равное 0, если вершины не соединены. Такая матрица называется матрицей смежности. Если в графе n вершин, то матрица смежности имеет размер $n \times n$.

Другой способ задания графа реализуется с помощью матрицы инцидентности. Ребро инцидентно вершине, если соединяет ее с какой-либо другой. Матрица инцидентности состоит из n строк (n – число вершин) и m столбцов (m – число ребер). Элемент данной матрицы a_{ij} принимает значение 1, если ребро j инцидентно вершине i, и значение равное 0, если не инцидентно.

Для анализа графов и извлечения из них необходимой информации разработаны различные методы и алгоритмы.

Одним из типов задач, решаемых с помощью графов, являются задачи поиска кратчайших путей между вершинами, построение сетей минимальной длины и т.п. Решение подобных задач основано на алгоритмах перебора вершин и ребер.

Для решения оптимизационных задач на графах с помощью языка Python мы будем использовать библиотеку NetworkX [16]. В данной библиотеке возможно создание различных типов графов с помощью функций:

а) `Graph()` – простой неориентированный граф, при этом узлы могут быть соединённые сами с собой;

б) `DiGraph()` – ориентированный граф;

в) `MultiGraph()` – мультиграф, в нем возможно существование пар вершин, которые соединены более чем одним ребром, либо более чем двумя дугами противоположных направлений;

г) `MultiDiGraph()` – ориентированный мультиграф.

Составляющими элементами любого графа являются вершины (`node`) и ребра (`edge`), которые имеют свои собственные идентификаторы, а также различные атрибуты.

В библиотеке NetworkX граф может быть создан тремя методами, отличающимися способами задания вершин и дуг: с использованием генератора графов с предустановленной топологией (полный граф, циклические графы и т.д.); загрузкой данных из структуры (матрица смежности, матрица инцидентности и др.); последовательным добавлением вершин (узлов) и (или) ребер.

Функция `add_node()` применяется для добавления одной вершины, а функция `add_nodes_from()`, аргументом которой является список, кортеж, строки из файла или узлы другого графа, применяется для добавления сразу нескольких вершин. Чтобы удалить вершину используется функция `remove_node()`.

Для добавления ребер между парой вершин применяется функция `add_edge()`, а для удаления ребра функция `remove_edge()`. Параметр `weight` функции `add_edge()` задает «вес» ребра. Данный параметр не является фиксированным атрибутом функции `add_edge()`, его значение и название задается пользователем. В алгоритмах библиотеки NetworkX, например, в алгоритмах для вычисления положения узлов при визуализации графа, используется фиксированный атрибут `weight`, которому по умолчанию присвоено имя `weight='weight'`. Поэтому, если для

названия «весов» ребер использовать какое-либо другое имя, то в алгоритмах, в которых присутствует атрибут `weight`, необходимо обязательно указывать выбранное имя `weight='имя'`.

Функция `degree('A', weight='weight')` возвращает сумму весов ребер непосредственно соединенных с вершиной «A». Если параметр `weight` не указан, то веса ребер считаются равными 1, и результатом будет число ребер, непосредственно соединенных с вершиной «A».

Чтобы визуализировать полученный граф, в библиотеке NetworkX существуют разные способы, например:

а) `draw()` – самый простой способ, без детальной настройки;

б) `draw_networkx()` – визуализация с дополнительными параметрами;

в) `draw_networkx_nodes()` – строит все узлы;

г) `draw_networkx_edges()` – строит все ребра;

д) `draw_circular()` – располагает узлы по кругу.

В примере 8.1 показано построение графа с вершинами «A», «B», «C», «D» и простой визуализацией с помощью функции `draw()`.

Пример 8.1. Создание и визуализация графа.

```
In[1]:
import networkx as nx

G = nx.Graph()
G.add_node('A')
G.add_node('B')
G.add_node('C')
G.add_node('D')
G.add_edge('A', 'B', weight = 2)
G.add_edge('A', 'D', weight = 3)
G.add_edge('C', 'D', weight = 2)
G.add_edge('B', 'C', weight = 2)
G.add_edge('B', 'D', weight = 1)

nx.draw(G,  node_size = [900, 900, 900, 900],
        font_size=22,
```

```
        with_labels=True,
        node_color="y")
ax = plt.gca()
# Задаем поля области построения, чтобы не
# обрезались узлы:
ax.margins(0.1)
```

Результат работы кода представлен на рисунке 8.1.

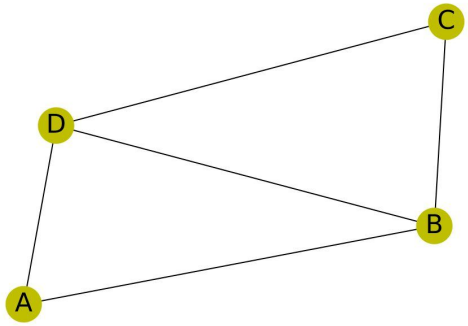


Рисунок 8.1 – Визуализация графа из примера 8.1

Этап добавления вершин можно пропустить и сразу перейти к добавлению ребер. Функция `add_edge('A','B')` автоматически добавит какую-либо из вершин «A» и (или) «B», если этой вершины еще нет.

По умолчанию функция `draw()` для расчета расположения узлов графа применяет алгоритм `spring_layout`. При этом при повторных запусках кода расположение вершин будет меняться. Чтобы зафиксировать их положение используется атрибут `pos` (представляет собой словарь), в который значения координат вершин передаются или вручную, или с помощью функции `get_node_attributes()`.

В примере 8.2 показано построение графа с заданными координатами вершин и его визуализацией с помощью метода `draw_networkx()`.

Пример 8.2. Построение взвешенного графа с фиксированным положением узлов.

```
In[2]:
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

G.add_node('A', coordinate = (0,0))
G.add_node('B', coordinate = (0.4,0.6))
G.add_node('C', coordinate = (1,0.5))
G.add_node('D', coordinate = (0.8,0.1))
G.add_edge('A', 'B', weight = 1)
G.add_edge('A', 'C', weight = 2)
G.add_edge('A', 'D', weight = 4)

pos = nx.get_node_attributes(G,'coordinate')

labels = nx.get_edge_attributes(G,'weight')

nx.draw_networkx(G,
                 pos,
                 node_size = [800, 800, 800, 800],
                 font_size = 20,
                 with_labels = True,
                 node_color = "y")

nx.draw_networkx_edge_labels(G, pos,
                             font_size=16,
                             edge_labels=labels)

ax = plt.gca()
ax.margins(0.1)
plt.axis("off")
```

Результат выполнения кода представлен на рисунке 8.2.

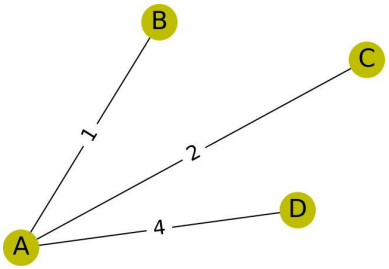


Рисунок 8.2 – Визуализация взвешенного графа с фиксированным положением вершин

8.2 Практическая работа

Цель работы: получить навыки решения оптимизационных задач на графах с помощью средств и методов языка Python и библиотеки NetworkX.

8.2.1 Пример выполнения работы

Решим следующую задачу.

Имеется 7 населенных пунктов, соединенных дорогами с известными расстояниями между соединенными городами. Требуется выполнить следующие задания:

а) найти кратчайший путь и его длину между городом №1 и городом №7;

б) проложить вдоль дорог волоконно-оптическую линию связи, соединяющую все города и имеющую минимальную длину.

Начинаем выполнение задания с импорта библиотеки для работы с графами NetworkX и создаем пустой граф `G`:

```
import networkx as nx
import matplotlib.pyplot as plt

G=nx.Graph()
```

В этот граф добавим вершины с их координатами из заранее созданного списка `Uzly`:

```
Uzly=[('1', {'koord':(0,0)}),
      ('2', {'koord':(0,1)}),
      ('3', {'koord':(0.5,0)}),
      ('4', {'koord':(0.5,1)}),
      ('5', {'koord':(1,0)}),
      ('6', {'koord':(1,1)}),
      ('7', {'koord':(1.5,0.5)})
]
```

Заранее создадим список (`Dlina`) ребер с их весами. Веса назовем `length`:

```
Dlina=[('1', '2', {'length':15}),
       ('1', '3', {'length':12}),
       ('2', '3', {'length':3}),
       ('2', '4', {'length':6}),
       ('2', '7', {'length':30}),
       ('3', '5', {'length':4}),
       ('4', '5', {'length':8}),
       ('4', '7', {'length':5}),
       ('5', '6', {'length':2}),
       ('6', '7', {'length':5})
]
```

Далее с помощью функций `add_nodes_from()` и `add_edges_from()` добавим из созданных списков в граф узлы и ребра:

```
G.add_nodes_from(Uzly)
G.add_edges_from(Dlina)
```

Чтобы положения вершин были зафиксированы, используем их координаты при визуализации графа функцией `draw_networkx()`. Для этого в данной функции необходимо указать

атрибут pos, содержащий координаты вершин. В данный атрибут координаты вершин передаем функцией get_node_attributes():

```
pos = nx.get_node_attributes(G, 'koord')
```

В атрибут labels запишем значения весов ребер:

```
labels = nx.get_edge_attributes(G,'length')
```

После этого визуализируем граф и для каждого ребра добавим подпись, показывающую его вес:

```
nx.draw_networkx(
          G,
          pos,
          node_size = [800, 800, 800, 800, 800, 800, 800],
          font_size=20,
          with_labels=True,
          node_color='y'
)

nx.draw_networkx_edge_labels(G,
                             pos,
                             font_size=16,
                             edge_labels=labels)
```

Далее для поиска кратчайшего пути между вершинами №1 и №7 применяем функцию shortest_path(), а для вычисления его длины – функцию shortest_path_length():

```
print('кратчайший путь между вершинами №1 и №7 = ',
      nx.shortest_path(G, '1', '7', weight='length'))

print('длина кратчайшего пути между вершинами №1 и №7 = ',
      nx.shortest_path_length(G,'1', '7',
      weight='length'))
```

```
ax = plt.gca()
ax.margins(0.1)
plt.axis("off")
```

В результате получим:

```
кратчайший путь между вершинами №1 и №7 = ['1', '3', '5',
'6', '7']
длина кратчайшего пути между вершинами №1 и №7 = 23
```

Получившийся граф представлен на рисунке 8.3.

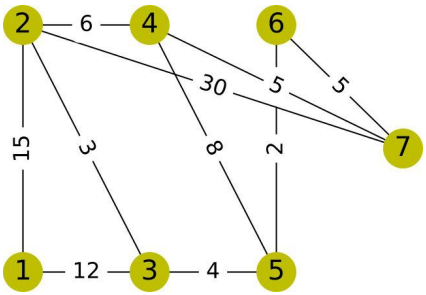


Рисунок 8.3 – Расположение городов и сеть дорог между ними

Построим теперь сеть минимальной длины. Для этого используем функцию `minimum_spanning_tree()`, в ней в качестве атрибута `weight` мы используем принятое нами обозначение весов ребер `length`:

```
G_min = nx.minimum_spanning_tree(G, weight = 'length')

nx.draw_networkx(
          G_min,
          pos,
          node_size = [800, 800, 800, 800, 800, 800, 800],
          font_size = 20,
          node_color = 'y'
)
```

```
labels_min = nx.get_edge_attributes(G_min, 'length')

nx.draw_networkx_edge_labels(G_min,
                             pos,
                             font_size = 16,
                             edge_labels = labels_min)

ax = plt.gca()
ax.margins(0.1)
plt.axis("off")
```

В результате получим сеть минимальной длины, которая представлена на рисунке 8.4.

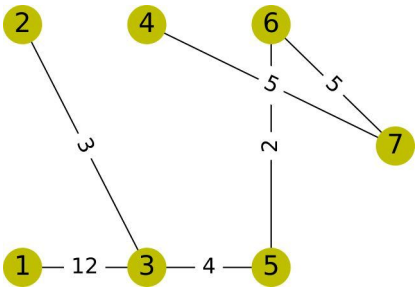


Рисунок 8.4 – Сеть минимальной длины

8.2.2 Задания для самостоятельной работы

В данной работе с помощью алгоритмов оптимизации на графах, представленных в библиотеке NetworkX, решается задача нахождения минимального пути и сети минимальной протяженности.

В ходе выполнения работы должны быть решены следующие задачи:

а) сформулировать словесный алгоритм программы;

б) построить блок-схему программы;

в) написать код программы;

г) сделать выводы и составить отчет.

Условие задачи для всех вариантов одинаковое: имеется 8 населенных пунктов: *A*, *B*, *C*, *D*, *E*, *F*, *G*, *H*. Населенные пункты соединены дорогами с известными расстояниями между соединенными городами. Требуется:

а) найти кратчайший путь и его длину между городами *E* и *F*;

б) проложить вдоль дорог волоконно-оптическую линию связи, соединяющую все города и имеющую минимальную длину.

Конкретные значения координат городов и длин дорог, соединяющих города, представлены в индивидуальных вариантах.

Индивидуальные варианты задания:

1. Координаты городов: $A=(0,0)$, $B=(3,0)$, $C=(0,6)$, $D=(5,5)$, $E=(3,9)$, $F=(6,1)$, $G=(2,5)$, $H=(8,8)$. Длины дорог: $S_{AB}=5$, $S_{BC}=10$, $S_{AC}=7$, $S_{DB}=8$, $S_{EG}=8$, $S_{EC}=9$, $S_{FH}=3$, $S_{CH}=9$, $S_{DF}=4$, $S_{DG}=3$, $S_{GH}=2$, $S_{GB}=2$.
2. Координаты городов: $A=(0,3)$, $B=(5,5)$, $C=(2,0)$, $D=(3,5)$, $E=(1,0)$, $F=(5,3)$, $G=(1,5)$, $H=(4,1)$. Длины дорог: $S_{AB}=5$, $S_{BC}=7$, $S_{AC}=12$, $S_{DF}=5$, $S_{EG}=8$, $S_{FG}=11$, $S_{FH}=7$, $S_{CH}=9$, $S_{AG}=14$, $S_{DG}=3$, $S_{GH}=2$, $S_{AC}=2$.
3. Координаты городов: $A=(1,0)$, $B=(0,4)$, $C=(1,7)$, $D=(3,0)$, $E=(3,4)$, $F=(6,1)$, $G=(6,5)$, $H=(3,6)$. Длины дорог: $S_{AB}=2$, $S_{BC}=4$, $S_{AC}=10$, $S_{DF}=5$, $S_{EG}=8$, $S_{FH}=7$, $S_{CH}=5$, $S_{AG}=14$, $S_{AE}=14$, $S_{BD}=3$, $S_{GH}=2$, $S_{AC}=2$.
4. Координаты городов: $A=(1,2)$, $B=(0,4)$, $C=(5,4)$, $D=(5,0)$, $E=(3,0)$, $F=(6,6)$, $G=(6,2)$, $H=(3,6)$. Длины дорог: $S_{AB}=2$, $S_{BC}=11$, $S_{AC}=8$, $S_{BF}=7$, $S_{CE}=9$, $S_{EG}=8$, $S_{FH}=5$, $S_{CH}=5$, $S_{AG}=14$, $S_{BD}=3$, $S_{DE}=2$, $S_{AC}=2$.
5. Координаты городов: $A=(4,5)$, $B=(0,4)$, $C=(5,1)$, $D=(3,0)$, $E=(0,2)$, $F=(6,6)$, $G=(6,2)$, $H=(2,6)$. Длины дорог: $S_{AB}=7$, $S_{BC}=11$, $S_{AF}=5$, $S_{DF}=12$, $S_{CE}=9$, $S_{FH}=6$, $S_{CH}=11$, $S_{FG}=4$, $S_{GF}=4$, $S_{BD}=3$, $S_{DE}=2$, $S_{AC}=2$.
6. Координаты городов: $A=(0,4)$, $B=(5,4)$, $C=(3,1)$, $D=(0,2)$, $E=(1,0)$, $F=(6,6)$, $G=(6,2)$, $H=(2,6)$. Длины дорог: $S_{BF}=3$,

$S_{BC}=11$, $S_{AF}=7$, $S_{AD}=12$, $S_{CE}=3$, $S_{FH}=6$, $S_{DH}=4$, $S_{CG}=4$, $S_{GF}=4$, $S_{BD}=3$, $S_{DE}=2$, $S_{AC}=2$.

7. Координаты городов: $A=(4,3)$, $B=(2,5)$, $C=(3,1)$, $D=(1,2)$, $E=(1,0)$, $F=(5,6)$, $G=(6,2)$, $H=(0,6)$. Длины дорог: $S_{AB}=12$, $S_{BC}=11$, $S_{AF}=7$, $S_{CD}=12$, $S_{CE}=3$, $S_{FH}=6$, $S_{DH}=4$, $S_{CG}=4$, $S_{GF}=4$, $S_{BD}=3$, $S_{DE}=2$, $S_{AC}=2$.

8. Координаты городов: $A=(0,0)$, $B=(2,1)$, $C=(0,6)$, $D=(6,4)$, $E=(3,9)$, $F=(7,0)$, $G=(2,5)$, $H=(8,7)$. Длины дорог: $S_{AB}=2$, $S_{BC}=6$, $S_{AC}=7$, $S_{BD}=5$, $S_{EG}=4$, $S_{EC}=9$, $S_{FH}=5$, $S_{CH}=15$, $S_{DF}=4$, $S_{DG}=3$, $S_{GH}=10$, $S_{GB}=2$.

9. Координаты городов: $A=(0,0)$, $B=(2,0)$, $C=(0,4)$, $D=(7,4)$, $E=(4,3)$, $F=(7,0)$, $G=(2,8)$, $H=(8,6)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{AC}=3$, $S_{BD}=8$, $S_{EG}=4$, $S_{EC}=4$, $S_{FH}=5$, $S_{CH}=15$, $S_{DF}=4$, $S_{DG}=11$, $S_{GH}=10$, $S_{GB}=7$.

10. Координаты городов: $A=(1,0)$, $B=(0,3)$, $C=(3,0)$, $D=(7,4)$, $E=(6,0)$, $F=(8,2)$, $G=(2,8)$, $H=(8,8)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{AC}=3$, $S_{BD}=8$, $S_{EG}=10$, $S_{EC}=4$, $S_{FH}=5$, $S_{CH}=15$, $S_{DF}=2$, $S_{DG}=11$, $S_{GH}=10$, $S_{GB}=7$.

11. Координаты городов: $A=(1,6)$, $B=(0,3)$, $C=(2,0)$, $D=(8,6)$, $E=(6,0)$, $F=(4,8)$, $G=(4,6)$, $H=(8,3)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{AC}=5$, $S_{BD}=8$, $S_{EG}=7$, $S_{EC}=4$, $S_{FH}=5$, $S_{CH}=15$, $S_{DF}=2$, $S_{AF}=11$, $S_{GH}=6$, $S_{GB}=7$.

12. Координаты городов: $A=(1,6)$, $B=(1,3)$, $C=(5,0)$, $D=(8,6)$, $E=(7,0)$, $F=(5,8)$, $G=(4,6)$, $H=(8,3)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{AC}=10$, $S_{BD}=12$, $S_{EG}=7$, $S_{EC}=4$, $S_{FH}=5$, $S_{CH}=5$, $S_{DF}=2$, $S_{AF}=7$, $S_{GH}=6$, $S_{GB}=7$.

13. Координаты городов: $A=(3,6)$, $B=(2,2)$, $C=(3,0)$, $D=(7,1)$, $E=(6,0)$, $F=(6,8)$, $G=(5,6)$, $H=(8,4)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{GD}=7$, $S_{BD}=12$, $S_{EG}=7$, $S_{EC}=4$, $S_{FH}=5$, $S_{CH}=6$, $S_{DF}=2$, $S_{AF}=7$, $S_{AG}=6$, $S_{GB}=3$.

14. Координаты городов: $A=(0,6)$, $B=(1,3)$, $C=(0,0)$, $D=(8,0)$, $E=(6,0)$, $F=(6,8)$, $G=(4,6)$, $H=(7,4)$. Длины дорог: $S_{AB}=2$, $S_{BC}=3$, $S_{GD}=6$, $S_{BD}=14$, $S_{EG}=6$, $S_{EC}=3$, $S_{FH}=5$, $S_{CH}=5$, $S_{DF}=3$, $S_{AF}=8$, $S_{AG}=5$, $S_{GB}=4$.
15. Координаты городов: $A=(0,6)$, $B=(4,6)$, $C=(0,0)$, $D=(7,2)$, $E=(6,0)$, $F=(6,8)$, $G=(0,3)$, $H=(8,4)$. Длины дорог: $S_{AB}=2$, $S_{BC}=8$, $S_{GD}=7$, $S_{BD}=4$, $S_{EG}=7$, $S_{EC}=6$, $S_{FH}=5$, $S_{CH}=11$, $S_{DF}=2$, $S_{AF}=7$, $S_{AG}=2$, $S_{GB}=3$.

8.3 Контрольные вопросы

1. Что такое граф?
2. Что такое матрица смежности?
3. Что такое матрица инцидентности?
4. Какой граф называется простым?
5. Какой граф называется пустым?
6. Какой граф называется полным?
7. Какой граф называется неориентированным?
8. Записать алгоритм построения минимального остова.
9. Записать алгоритм нахождения минимального пути между двумя вершинами неориентированного графа.
10. Какой граф называется ориентированным?
11. Какой граф называется мультиграфом?
12. Что такое минимальный остов?